



Práctica 02: Relación Muchos a Muchos



Objetivo

Implementar una relación **N:M** entre `Post` y `Tag` usando una tabla asociativa.



Escenario

- Un **Post** puede tener muchos **Tags**
- Un **Tag** puede estar en muchos **Posts**

Post (N) ———< post_tags >———— Tag (M)



Instrucciones

Paso 1: Crear la Tabla Asociativa

Abre `starter/models.py` y descomenta la tabla `post_tags` :

```
from sqlalchemy import Table, Column, Integer, ForeignKey

# Tabla asociativa (no es un modelo, solo una Table)
post_tags = Table(
    "post_tags",
    Base.metadata,
    Column("post_id", Integer, ForeignKey("posts.id"), primary_key=True),
    Column("tag_id", Integer, ForeignKey("tags.id"), primary_key=True)
)
```

Puntos clave:

- Es una `Table` , no una clase `Base`
- Tiene dos FKs que juntas forman la PK compuesta
- No tiene modelo ORM asociado (es transparente)

Paso 2: Crear el Modelo Tag

Descomenta la clase `Tag` :

```
class Tag(Base):
    __tablename__ = "tags"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(50), unique=True)

    # Relación N:M con Post
    posts: Mapped[list["Post"]] = relationship(
        secondary=post_tags,          # ← Usa la tabla asociativa
        back_populates="tags"
    )
```

Paso 3: Actualizar el Modelo Post

Añade la relación con tags al modelo Post:

```
class Post(Base):
    # ... campos existentes ...

    # Relación N:M con Tag
    tags: Mapped[list["Tag"]] = relationship(
        secondary=post_tags,
        back_populates="posts"
    )
```

Paso 4: Crear Tablas

En `starter/main.py`, descomenta:

```
Base.metadata.create_all(bind=engine)
```

Paso 5: Insertar Tags

Descomenta `create_tags()` :

```
def create_tags():
    db = SessionLocal()

    tag_names = ["python", "fastapi", "sqlalchemy", "tutorial", "backend"]

    for name in tag_names:
        tag = Tag(name=name)
        db.add(tag)

    db.commit()
    print(f"✅ Tags creados: {tag_names}")
    db.close()
```

Paso 6: Asignar Tags a Posts

Descomenta `assign_tags_to_posts()` :

```
def assign_tags_to_posts():
    db = SessionLocal()

    # Obtener un post
    post = db.execute(select(Post).limit(1)).scalar()

    # Obtener tags
    python_tag = db.execute(
        select(Tag).where(Tag.name == "python")
    ).scalar()
    fastapi_tag = db.execute(
        select(Tag).where(Tag.name == "fastapi")
    ).scalar()

    # Asignar tags al post (como si fuera una lista)
```

```

post.tags.append(python_tag)
post.tags.append(fastapi_tag)

db.commit()
print(f"✅ Tags asignados a '{post.title}': {[t.name for t in post.tags]}")
db.close()

```

Paso 7: Consultar Relaciones N:M

Descomenta `query_many_to_many()` :

```

def query_many_to_many():
    db = SessionLocal()

    # Posts de un tag específico
    python_tag = db.execute(
        select(Tag).where(Tag.name == "python")
    ).scalar()

    print(f"\n📝 Posts con tag '{python_tag.name}':")
    for post in python_tag.posts:
        print(f"    - {post.title}")

    # Tags de un post
    post = db.execute(select(Post).limit(1)).scalar()
    print(f"\n🏷️ Tags del post '{post.title}':")
    for tag in post.tags:
        print(f"    - {tag.name}")

    db.close()

```

Paso 8: Agregar/Eliminar Tags

Descomenta `manage_tags()` :

```

def manage_tags():
    db = SessionLocal()

    post = db.execute(select(Post).limit(1)).scalar()

    # Agregar tag
    tutorial_tag = db.execute(
        select(Tag).where(Tag.name == "tutorial")
    ).scalar()

    if tutorial_tag not in post.tags:
        post.tags.append(tutorial_tag)
        print(f"✅ Tag 'tutorial' agregado")

    # Eliminar tag
    for tag in post.tags:
        if tag.name == "fastapi":
            post.tags.remove(tag)
            print(f"❌ Tag 'fastapi' eliminado")

```

```
break
```

```
db.commit()
print(f"👉 Tags actuales: {[t.name for t in post.tags]}")
db.close()
```

✅ Checklist de Verificación

- ☐ La tabla `post_tags` se crea correctamente
- ☐ Puedo crear Tags independientes
- ☐ Puedo asignar Tags a Posts con `.append()`
- ☐ Puedo eliminar Tags de Posts con `.remove()`
- ☐ Puedo acceder a `post.tags` y `tag.posts`

🎯 Reto Extra

1. Crea una función `get_or_create_tag(name)` que retorne el tag si existe o lo cree si no
2. Implementa una función para encontrar posts que tengan TODOS los tags especificados
3. Cuenta cuántos posts tiene cada tag

[← Anterior: Práctica 01](#) | [Siguiente: Práctica 03 →](#)